

# HJ-SAC Safety Filter — Change List

cross\_embodiment/CEHJ · every row states the current behaviour and the intended replacement. Line references are to the code as it stands today. **Part I** is the training-pipeline change list. **Part II** is a separate set of frozen-encoder input mismatches found later by scanning HoloBrain in RoboOrchardLab; those are silent and more damaging, and should be fixed first even though they appear second.

Headline settings: h = obstacle distance only, never infinite · 25 Hz control (exactly 10 physics steps per tick) · single 1 cm margin · 3-tick HJ blocks (0.12 s) · 4500 physics steps before timeout · 5 tasks x 4 embodiments · actor bounded to 25% of joint speed limit ( $\kappa = 0.25$ ) over each 40 ms tick.

**Prerequisite (not a code change).** CKPT\_DIR in main/train/collect.py and tests/test\_encoder\_step.py points at /root/autodl-tmp/HoloBrain/HoloBrain\_v0.0\_GD, an AutoDL path that returns permission denied on this cluster; likewise /root/autodl-tmp/RoboTwin/... in tests/gen\_franka\_dualarm\_urdf.py. These need real paths before collection can run.

## PART I — Training-pipeline changes (h, control rate, actor execution, switching, collection, buffer, logging)

### 1. Hazard function — main/train/hfunc.py, main/envs/obstacle.py

h is the safety specification the value function learns, so this is the change everything else is calibrated against. It must be finite on every stored transition.

| Item                                              | Previously                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>h definition</b>                               | <code>h = min(d_block, d_table)</code> . <code>d_block = distance_info(env)["d_system"]</code> is arms plus held payload against the obstacle OBB. <code>d_table</code> is a separate half-space term: the minimum over arm collision spheres of <code>center_z - radius - table_z - table_margin</code> , with <code>table_z = table_height + task.table_z_bias</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <code>h = d_block</code> <b>only</b> . Because <code>d_system == d_min == min(d_left, d_right, d_left_held, d_right_held)</code> in <code>distance.py</code> , <code>h</code> becomes exactly the <code>d_min</code> that <code>run_safe_unsafe_bimanual_plank.sh</code> already reports. Training <code>h</code> and the sweep's distance are then the same number, directly comparable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Table term removal and the spawn guarantee</b> | <code>compute_h</code> builds <code>left_spheres / right_spheres</code> from <code>_arm_spheres</code> , computes <code>d_table</code> , and folds it into both <code>h</code> and every entry of <code>per_link</code> ; <code>main/envs/distance.py</code> has no table logic at all, which is why the sweep never saw this term. Separately, <code>d_block = inf</code> when <code>env.obstacle</code> is <code>None</code> , but <code>d_table</code> was always finite so <code>h</code> stayed finite. <code>geometric_pose</code> tries the corridor <code>t</code> , then perpendicular nudges of 0.03 to 0.16 m, then a best-line-distance fallback; if all fail it prints "no spawn pose clears pick/target keepaway" and returns <code>(None, None)</code> , <code>choose_and_spawn</code> passes that up as <code>(None, None, None, arm)</code> , and the episode runs obstacle-free. | Delete the table computation from <code>compute_h</code> . <code>table_margin</code> and <code>table_height</code> stay in <code>FrozenConfig</code> because <code>update_curobo_world</code> still pushes the table into cuRobo's world model, so the planner keeps avoiding it; only the value function stops scoring it. <b>That makes the spawn path safety-critical:</b> with nothing else bounding <code>h</code> , an obstacle-free episode gives <code>h = +inf</code> on every tick, which reaches the memmap and turns the Bellman target into NaN on the first batch that samples it. So spawning becomes total, in three tiers. <b>Tier 1</b> is the existing chain, unchanged. <b>Tier 2 is new:</b> if <code>on_path</code> placement fails, place the obstacle anywhere feasible on the table by scanning the existing <code>_table_grid(block_r)</code> lattice — 11 x 9 points already inset by <code>block_r + 0.02</code> — keeping candidates that <code>pass_in_table</code> and <code>_keepaway_ok</code> and taking the one with maximum keepaway clearance. <b>Tier 3:</b> if that set is empty, skip the episode. <code>collect.py</code> tests the returned actor for <code>None</code> and re-draws from the schedule instead of proceeding. |
| <b>per_link and true_argmin</b>                   | <code>per_link[label] = min(obstacle_distance, d_tab)</code> , so <code>true_argmin</code> could name a link that is merely low over the table rather than near the obstacle.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Obstacle distance only, so <code>true_argmin</code> always names the link genuinely closest to the box. That makes it a usable check on the learned per-token <code>argmin</code> in the <code>softmin</code> readout.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Arm-to-arm distance</b>                        | <code>d_arm_arm</code> computed over all left/right sphere pairs and reported in <code>diag</code> , but deliberately excluded from <code>h</code> ; self-collision is out of scope.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Unchanged.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Accepted risk</b>                              | The table term was the only thing preventing the actor from treating the tabletop as free space.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | The HJ actor has no collision model and is bounded only by <code>d0_max</code> , so nothing now penalizes driving a link into the table, particularly in early rounds when the policy is close to random. Accepted on the grounds that the obstacle is lateral and descending does not increase obstacle clearance. Tracked by the diagnostic in section 8 rather than by <code>h</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

### 2. Config — main/train/config.py

The first two rows are coupled: `control_freq`, `control_dt` and `delta_theta_max` have to move together.

| Item                                                       | Previously                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>control_freq</b>                                        | <code>Env(control_freq=20.0)</code> . 250 / 20 = 12.5, so <code>_physics_substeps</code> carries <code>_substep_acc</code> and emits 12 or 13 physics steps per tick, alternating; <code>RolloutController.hooked_step</code> mirrors it with <code>_tick_acc += 0.08</code> . Every index calculation has to consult the accumulator to know how long the current tick is. A 2720-physics-step episode is 218 control ticks.                                                                                                                                                                                                                                                                                                                                                                                                                                        | 25.0 Hz. 250 / 25 = 10 exactly, so a control tick is always exactly 10 physics steps and neither accumulator ever carries a fraction. <b>That is the main reason to do it:</b> <code>a_nom</code> spans a fixed 10 plan rows, tick <code>m</code> starts at physics step <code>10 * m</code> , and the 12-or-13 ambiguity leaves the action-index logic entirely. Cost: the same episode is 272 ticks, so 25% more encoder forwards and 25% more stored transitions — at roughly 630 KB per step, about 171 MB per episode rather than 137 MB.                                                                                                                                                                                                                        |
| <b>The 20 Hz literals that control_freq does NOT reach</b> | Seven call sites hardcode the old rate instead of reading it from the env, so <code>control_freq</code> is <b>not</b> a single knob. Defaults <code>control_freq: float = 20.0</code> at <code>main/train/collect.py:113</code> , <code>main/train/nominal.py:129</code> (probe) and <code>:184</code> (cached), and <code>main/envs/env.py:337</code> . Literal conversions <code>int(cfg_ep.max_steps_per_episode * 12.5)</code> at <code>main/train/collect.py:185</code> and <code>main/train/eval_utils.py:63</code> , and <code>round(int(n_chunk) / 12.5)</code> at <code>main/train/rollout.py:251</code> . Worst of the set is <code>main/train/nominal.py:220</code> , <code>step_advance = physics_freq / 20.0</code> , which <b>ignores the control_freq argument its own function accepts</b> — latent today only because the default happens to be 20. | Change all seven to derive from <code>env.control_freq</code> (or <code>cfg.control_dt</code> ); the three 12.5 literals become <code>PHYSICS_FREQ / control_freq</code> , which evaluates to exactly 10. <b>Grep for 12.5 and 20.0 before declaring the rate change done</b> — editing <code>FrozenConfig.control_dt</code> alone leaves the planner advancing 12.5 path rows per 10-step tick, i.e. the nominal would run 25% fast against a correct clock and <code>a_nom</code> would no longer match what the arm was commanded. Two sites are already correct and need no edit: <code>main/train/rollout.py:317</code> ( <code>env.control_freq / env.PHYSICS_FREQ</code> ) and <code>main/envs/env.py:512</code> ( <code>PHYSICS_FREQ / control_freq</code> ). |
| <b>control_dt and delta_theta_max</b>                      | <code>control_dt = 0.05</code> s, matching 20 Hz. <code>delta_theta_max = joint_vel_limits * control_dt * kappa</code> with <code>kappa = 0.25</code> , so 0.0125 x the velocity limit per tick.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <code>control_dt = 0.04</code> s, which must move in the same commit as <code>control_freq</code> since it feeds <code>delta_theta_max</code> in <code>body_features.py</code> and the <code>softmin / Jacobian</code> scaling — a mismatch would silently misbound the actor. The formula is unchanged, so the bound shrinks 20% to 0.01 x the velocity limit at unchanged <code>kappa</code> . Either accept the smaller per-tick action — finer control, but about 25% more ticks to cover the same motion — or raise <code>kappa</code> to 0.3125 to hold physical displacement constant. Recommend accepting the smaller bound, since the extra ticks are already being paid for.                                                                                |
| <b>max_steps_per_episode</b>                               | 400 control ticks, converted at the call site as <code>int(400 * 12.5) = 5000</code> physics steps, or 20 s of simulated time.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 450 control ticks, giving <code>450 * 10 = 4500</code> physics steps, or 18 s — now an exact integer conversion. Enforced in <code>RolloutController.hooked_step</code> , which counts <code>scene.step()</code> calls and raises <code>RolloutTimeout</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>hj_hold_ticks</b>                                       | Did not exist as a field. Intervention length was implicitly <code>round(replan_k / 12.5) = 5</code> control ticks, derived from the 60-step window rather than chosen.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | New, 3. At 25 Hz that is 0.12 s, exactly 30 physics steps.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>obstacle_model / obstacle_choices</b>                   | "086_woodenblock" default, sampled per episode from <code>( "086_woodenblock", "059_pencup" )</code> — a 10.3 cm cube and a 9.8 x 11.7 x 9.8 cm cup. Two geometries mean <code>h</code> is measured against a different OBB from episode to episode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Fixed "068_boxdrink", a 11.0 x 15.4 x 11.6 cm box, with <code>obstacle_choices = ( "068_boxdrink", )</code> . Matches <code>--obstacle-model 068_boxdrink</code> in the sweep script, so training and the reference runs measure against identical geometry. It is taller than either previous option at 15.4 cm standing, so the arm can no longer pass over it as easily.                                                                                                                                                                                                                                                                                                                                                                                           |

| Item                       | Previously                                                                                                                                                                                                                                                                                                                                                      | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>task_choices</b>        | Eight tasks: <code>stack_blocks_two</code> , <code>stack_bowls_two</code> , <code>grab_roller</code> , <code>pick_dual_bottles</code> , <code>place_bread_basket</code> , <code>place_burger_fries</code> , <code>place_can_basket</code> , <code>place_cans_plasticbox</code> .                                                                                | Five tasks: <code>stack_blocks_two</code> , <code>stack_bowls_two</code> , <code>place_burger_fries</code> , <code>place_bread_basket</code> , <code>place_can_basket</code> . All five have explicit corridors entries in TASK_SPECS, so on/off-path spawning is well defined. Note the dropped <code>grab_roller</code> has <code>object == target == "roller"</code> , making its corridor degenerate and forcing <code>geometric_pose</code> onto a hardcoded fallback direction.                     |
| <b>embodiment_choices</b>  | Four: <code>piper</code> , <code>franka-panda</code> , <code>ARX-X5</code> , <code>ur5-wsg</code> . Three are 6-DOF per arm; <code>franka-panda</code> is 7-DOF, which is what drives the buffer padding.                                                                                                                                                       | Unchanged.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>obstacle_t</b>          | Hardcoded 0.6, so the obstacle always sits at the same 60% point of the pick-to-place corridor. Some incidental spread exists when <code>_t_bounds</code> snaps <code>t</code> away from a keepaway, but the preferred value never varies.                                                                                                                      | Sampled per episode with <code>_corridor_t(seed)</code> from <code>main/envs/run.py</code> : <code>np.random.RandomState(seed + 917).uniform(0.30, 0.70)</code> . Reusing that exact function keeps parity with the sweep, where corridor position produces the <code>t37 / t42 / t52 / t63</code> spread in the output directory names. Bounds match <code>CORRIDOR_T_LO = 0.30</code> and <code>CORRIDOR_T_HI = 0.70</code> .                                                                           |
| <b>obstacle_mode</b>       | "on_path" for every episode.                                                                                                                                                                                                                                                                                                                                    | 80% <code>on_path</code> , 20% <code>off_path</code> , drawn per episode. <code>off_path</code> places the obstacle roughly 0.22 m lateral to the corridor, so the expert usually stays clear and <code>h</code> stays large — those episodes supply the safe side of the value function. Expect the unsafe fraction of the buffer to drop by about a fifth.                                                                                                                                              |
| <b>h_scale</b>             | 20.0. <code>h</code> is stored and trained in units of 20 x metres.                                                                                                                                                                                                                                                                                             | 100.0, so <code>h</code> and <code>V</code> read directly in centimetres. <code>softmin_T</code> and the filter margin stay in physical metres in the config and are multiplied by <code>h_scale</code> at the point of use, so physical sharpness is preserved automatically. <b>Watch the gradients:</b> Bellman targets grow 5x, so critic gradients grow roughly 5x against the fixed <code>clip_grad_norm(..., 10.0)</code> and that clip will bind far more often. Monitor <code>gn_critic</code> . |
| <b>Filter margin</b>       | Two values: <code>margin_on = 0.0</code> and <code>margin_off = 0.005</code> metres, scaled to 0.0 and 0.1 in trained units. Engage when <code>V &lt; 0</code> , release when <code>V &gt; 0.5</code> cm. <code>margin_on = 0</code> means the filter waits until the safe policy itself can no longer hold clearance, leaving zero tolerance for critic error. | One margin at 0.01 m, i.e. 1.0 in the new cm units. Engage when <code>Q(s, a_nom) &lt; 1</code> cm. The <code>margin_off</code> field is deleted rather than set equal, so the code does not imply a hysteresis band that no longer exists.                                                                                                                                                                                                                                                               |
| <b>replan_k</b>            | 60 physics steps, which is 0.24 s at 250 Hz, per receding-horizon window. Part of the nominal's definition.                                                                                                                                                                                                                                                     | Removed; the nominal becomes vanilla <code>play_once</code> . Planning was never the cost driver — a vanilla episode spends 1.24 s planning out of 53.5 s total, against 43 s of rendering. The real saving is that vanilla episodes are shorter in physics steps, and the dominant per-episode cost in training is one encoder forward per control tick.                                                                                                                                                 |
| <b>n_episodes</b>          | 25 per run. With 20 (task, embodiment) combinations that gives 20 combos once and 5 twice.                                                                                                                                                                                                                                                                      | A multiple of 20 per collection round so the cyclic schedule closes evenly. 25 was also thin for 20 combinations; sizing is your call, but 200 episodes is roughly 57 GB at the new tick rate.                                                                                                                                                                                                                                                                                                            |
| <b>Other config fields</b> | <code>kappa = 0.25</code> , <code>softmin_T = 0.02</code> m, <code>gamma</code> annealing 0.9 to 0.999 over 50k steps, <code>alpha</code> 0.2 to 0.0 over 20k, <code>perturb_prob = 0.05</code> , <code>batch_size = 64</code> , <code>lr = 3e-4</code> , <code>tau = 0.005</code> , <code>rgbsd_archive_every = 25</code> .                                    | Unchanged. Note <code>gamma</code> 's effective horizon of about 1000 decision steps is 40 s at 25 Hz rather than 50 s at 20 Hz, still comfortably longer than an 18 s episode.                                                                                                                                                                                                                                                                                                                           |

### 3. Actor execution: turning one 25 Hz 60 into 10 physics steps — main/envs/env.py

The actor predicts a displacement over a 40 ms tick, but the environment currently applies it as a held position target with zero commanded velocity, so only about a fifth of it happens. This section is self-contained and can be implemented independently of everything else.

| Item                                           | Previously                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Timescales</b>                              | Physics is 250 Hz ( <code>Env.PHYSICS_FREQ</code> , <code>main/envs/env.py</code> :324). The control loop ran at 20 Hz, so one tick was 12 or 13 physics steps depending on <code>_substep_acc</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Physics 250 Hz, actor 25 Hz, so one action lasts exactly <b>N = 250 / 25 = 10 physics steps = 40 ms</b> . The network is never asked for a 4 ms twitch: it outputs a displacement spanning the whole 40 ms.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>What the actor predicts</b>                 | <code>TokenActor</code> head is Gaussian + tanh, and $\delta\theta = \tanh(\cdot) \times d\theta_{\text{max}}$ with <code>dθ_max = q̇_lim · Δt_ctrl · κ</code> , <code>κ = 0.25</code> ( <code>main/network/body_features.py</code> ). The bound is structural — applied inside the head — not a clip bolted on afterwards.                                                                                                                                                                                                                                                                                                                                    | Unchanged in form; only <code>Δt_ctrl</code> moves 0.05 → 0.04 s. So <b> δθ  ≤ 25% of the URDF joint speed limit, sustained over 40 ms</b> . That 25% figure is the actor's entire authority budget, which is why losing 80% of it to the drive (next row) is not survivable.                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Expanding 60 to the 250 Hz drives</b>       | <code>Env.step()</code> writes <code>θ_d = q + 60</code> <b>once</b> with <code>θ_d = 0</code> — the hardcoded zeros handed to <code>set_arm_joints</code> at <code>main/envs/env.py</code> :563 and :565 — then runs all 10 physics steps without rewriting the target ( <code>env.py</code> :567-569). Drive gains are <code>kp = 1000</code> , <code>kd = 200</code> ( <code>RoboTwin/assets/embodiments/*/config.yml</code> ), applied in <code>RoboTwin/envs/robot/robot.py</code> :625-633. The dominant pole is <code>kp/kd = 5 rad/s</code> ( $\tau \approx 0.2$ s), so the damper fights the spring and only <b>~20% of 60 is realized per tick</b> . | Interpolate the same $\delta\theta$ across the 10 steps instead of holding it. For <code>k = 1 ... N</code> : <b><math>\theta_d(k) = q + 60 \cdot k/N</math></b> and <b><math>\theta_d = 60 / \Delta t_{\text{ctrl}} = 25 \cdot 60</math></b> , the same velocity on all 10 steps; each <code>scene.step()</code> receives the pair ( <code>θ_d(k)</code> , <code>θ_d</code> ). The PD then tracks a short straight-line joint move and <b>commanded 60 ≈ achieved Δq</b> . Implementation: <code>Env.step()</code> must move <code>set_arm_joints</code> inside the substep loop rather than calling it once before it.                                                                                              |
| <b>Consistency with the nominal</b>            | The nominal executor already passes <code>CuRobo</code> 's planned velocity: <code>task.robot.set_arm_joints(pos[i], vel[i], "left")</code> at <code>main/envs/controller.py</code> :817 and :820. So the actor path is the <b>only</b> place in either codebase that sends a zero velocity target, and <code>a_nom</code> and $\pi(s)$ are not the same physical quantity today.                                                                                                                                                                                                                                                                              | After the fix both controllers issue (position, velocity) pairs at 250 Hz with identical semantics, so a single critic can score both. This is a <b>precondition</b> for the <code>Q(s, a_nom)</code> trigger and for storing commanded actions in the buffer — without it the critic learns a 5x-wrong action-to-effect map on the actor branch only.                                                                                                                                                                                                                                                                                                                                                                |
| <b>Why the network stays at 25 Hz</b>          | The 20 Hz rate was inherited from the old receding-horizon window rather than derived from anything.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <code>Q(s, a)</code> and <code>κ</code> are both <b>defined on the control tick</b> , so the tick is the MDP timestep. Predict 60 at 25 Hz and expand it to 250 Hz with ( <code>θ_d</code> , <code>θ_d</code> ). Do <b>not</b> make the actor emit 250 Hz waypoints — <code>κ</code> and <code>dθ_max</code> lose their meaning. Do <b>not</b> run TOPP to the goal: variable-length actions break the fixed <code>Δt</code> , and with it <code>γ</code> , the <code>a_nom</code> index math, and uniform <code>h</code> sampling.                                                                                                                                                                                   |
| <b>Reference: how RoboTwin expands actions</b> | Not consulted previously.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | For comparison, <code>RoboTwin/envs/_base_task.py</code> :1536-1560 builds the 2-point path [ <code>q_current</code> , <code>q_target</code> ], runs TOPP( <code>path</code> , <code>1/250</code> ), and executes <b>every</b> sample with <code>set_arm_joints(pos[i], vel[i])</code> until the segment completes (:1633-1660). Same principle — never a zero velocity target — but variable duration, which is exactly why we interpolate a fixed 10 steps instead of adopting TOPP. Note their silent failure path: if TOPP raises, <code>topp_flag</code> goes <code>False</code> , <code>n_step</code> is forced to 50 and <code>set_arm_joints</code> is never called, so the arm freezes for 50 physics steps. |

### 4. Switching — main/train/filter.py, main/train/rollout.py

| Item                    | Previously                                                                                                                                                                                                                                                                                                                                                              | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trigger quantity</b> | <code>Q(s, π_safe(s))</code> — the critic evaluated at the safe actor's own deterministic action. Both <code>SafetyFilter.value</code> and <code>.action</code> call <code>self.actor(..., deterministic=True)</code> and score that. Semantically this asks whether the safe policy can rescue the robot, which only becomes false once that policy is already losing. | <code>Q(s, a_nom)</code> — the critic evaluated at the planner's action for the upcoming tick. Semantically: if the planner does what it wants and the robot then behaves optimally safely forever, does it stay clear. This is the textbook least-restrictive filter and overrides only when the nominal action specifically is the problem. <code>SafetyFilter.action</code> already accepts an <code>a_nom</code> argument; it is currently passed <code>None</code> and only echoed back, never scored. |

| Item                                        | Previously                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Threshold and release                       | Two thresholds forming a latch: set engaged = True when $v < \text{margin\_on}$ , clear it when $v > \text{margin\_off}$ , both scored on $Q(s, \pi_{\text{safe}})$ . The 0.5 cm dead band stopped the filter flipping at every decision point when $V$ hovered at the boundary.                                                                                                                                                                                                              | One threshold at 1 cm with the latch removed: engage when $Q(s, a_{\text{nom}}) < 1$ cm, and at each 3-tick block boundary resume the nominal if $Q(s, a_{\text{nom}}) \geq 1$ cm, otherwise run another block. Chattering is prevented structurally by the block rather than by hysteresis, but at 3 ticks that block is only 0.12 s and is a weak substitute — log mode-switch count per episode as a chatter check, and lengthen the block rather than reinstating a second margin if it is high. <b>Cost caveat worth designing around:</b> the release test needs $a_{\text{nom}}$ , hence a plan, and a real cuRobo plan is about 96 ms of wall time, so replanning at every boundary during a sustained intervention is roughly 8 plans per simulated second. Cheaper scheme — score the release test against $a_{\text{nom}}$ formed from the existing plan, stale though it is, and pay for a genuine replan only once the test passes and control is actually handed back.                                                                                                                                                                        |
| Intervention block                          | 5 control ticks, 0.25 s, computed as $\text{round}(n_{\text{chunk}} / 12.5)$ from the 60-step window, so the length was tied to the nominal's replan period rather than chosen. One $\delta\theta$ was computed at the window start and reissued for all 5 ticks via <code>for _ in range(...): self.env.step_dtheta(a)</code> .                                                                                                                                                              | 3 control ticks, 0.12 s, exactly 30 physics steps, repeated until $Q(s, a_{\text{nom}})$ clears the margin. No cap, per your decision, so a stuck actor holds control until the 4500-step timeout fires; that outcome is logged distinctly so it does not masquerade as an ordinary timeout. The actor is re-queried every tick from a fresh observation — new encoder forward, new trunk pass, new $\delta\theta$ — since reissuing one displacement across the block would be $3 \times d\theta_{\text{max}}$ of open-loop motion with no feedback. Three encoder forwards per block is cheap, and each one also yields the $Q$ value the video panel needs.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Vanilla nominal: hook point and stale plans | All three modes — <code>_filtered_play_seq</code> , <code>_shadow_play_seq</code> , <code>_collect_play_seq</code> — hook in by overriding <code>PlanEveryKController._play_seq</code> , which fires once per 60-step window. Staleness never arose, because the receding-horizon controller called <code>_after_chunk</code> after every window, replanning from the live <code>qpos</code> toward the stored <code>_cehj_target_pose</code> , so any divergence was absorbed within 0.24 s. | <code>VanillaPlayOnceController</code> is a bare subclass of <code>ResidualController</code> with no <code>_play_seq</code> ; stock <code>take_dense_action</code> plays an entire spline in one call with no window boundaries. A new tick-chunked wrapper around it becomes the hook: it walks the spline in fixed 10-step chunks, forms $a_{\text{nom}}$ , evaluates $Q$ , and either continues or diverts. All three modes migrate onto it. The wrapper also has to handle the staleness vanilla now creates: it plans once per Action and reports <code>n_replans: 0</code> , so after the actor moves the arm the remaining waypoints are stale and <code>set_arm_joints(pos[i], ...)</code> <b>snaps</b> the drive targets back to a configuration the arm has left — undoing the avoidance maneuver, and entering the buffer as an action many multiples of $d\theta_{\text{max}}$ . The fix reuses existing machinery: treat an intervention as a forced <code>_after_chunk</code> , discarding the remaining waypoints and replanning the same target from live <code>qpos</code> , carrying <code>grip_i</code> across rather than resetting it. |
| Shadow-V mode                               | <code>_shadow_play_seq</code> runs the filter as a passive observer in nominal mode: it evaluates <code>self.flt.value(...)</code> , records $V$ , hardcodes <code>intervened = False</code> , and always falls through to the nominal. This preserves the counterfactual — a filter that acts prevents the collision it was supposed to predict.                                                                                                                                             | Switch <code>filter.value</code> to $Q(s, a_{\text{nom}})$ as well, so the eval figure plots exactly the quantity being thresholded. Leaving it alone would silently validate a different number. The two are not interchangeable: $\pi_{\text{safe}}$ approximately maximizes $Q$ , so $Q(s, a_{\text{nom}})$ is uniformly the smaller, and the filter fires more often at the same 1 cm margin than the old curve suggests. $a_{\text{nom}}$ is available for free here since the plan dicts are already passed into the hook.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

## 5. Collection — main/train/collect.py, main/train/rollout.py

| Item                                                | Previously                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Behavior policy                                     | The RoboTwin scripted <code>play_once</code> stage machine with all motion planned by cuRobo, plus a 5% chance per window of one control tick of uniform random $\delta\theta$ within $\pm d\theta_{\text{max}}$ . The HJ actor is <b>never constructed</b> — <code>collect()</code> builds <code>RolloutController</code> with no actor and no critics, so <code>self.flt</code> is <code>None</code> . Nothing learned influences the data and the buffer contains zero on-policy samples.                                                                                                                                                                                                                              | 80% of episodes run the full deployment stack, vanilla nominal plus HJ switching at 1 cm; 20% run nominal only. The dataset then covers the state distribution the deployed filter actually induces, which fixes the coverage gap where the critic must currently extrapolate to the actor's action region from planner-scale data.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Pipeline structure                                  | Strictly sequential and single-pass: <code>collect.py</code> runs all episodes to completion and writes the memmap, then <code>train.py</code> reads that fixed buffer and never touches the simulator except at eval.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Alternating rounds. Round 0 must be nominal-only since the actor is randomly initialized at that point. Then train, then each subsequent round collects with current weights and appends. This turns <code>train.py</code> from a single offline loop into an outer DAGger-style loop and requires the encoder, <code>policy_enc</code> , actor and critics to be live during collection.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Action stored, and why it matters                   | Achieved displacement: <code>qpos_raw[t+1] - qpos_raw[t]</code> , read back from the simulator at consecutive captures. Applied uniformly — <code>_collect_tick</code> never asks which controller drove the tick, so planner ticks, actor ticks and perturbation ticks are all handled by the same three lines. On contact the achieved displacement collapses toward zero while the command stays large, so the buffer records a near-zero action ending in collision and the critic learns that holding still is dangerous there — the wrong causal attribution, in exactly the transitions carrying the safety signal. In the reference sweep 53 of 64 vanilla episodes make contact, so this is the dominant regime. | Commanded displacement, per source. Planner ticks store the net delta-joints the spline asks for over the tick; HJ ticks store exactly the $\delta\theta$ handed to <code>step_dtheta</code> ; perturbation ticks store the sampled $\delta\theta$ , which already exists in a local variable and is currently discarded. Requires the executor to report its command, which the new tick wrapper is positioned to do. This attributes the collision to the action that caused it, and matches what $Q$ is queried with at deployment for both $Q(s, a_{\text{nom}})$ in the filter and $Q(s, \pi(s))$ in the actor loss.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Building a_nom: reference point and indices         | Neither arose. Achieved displacement is measured-to-measured by construction, and at 20 Hz any index arithmetic would have been awkward anyway, since a tick spanned 12 or 13 plan rows depending on where <code>_tick_acc</code> happened to land.                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | $a_{\text{nom}} = \text{pos}[i_{\text{end}}] - \text{qpos\_now}$ , referenced against the robot's <b>live</b> joint state rather than the plan's own starting waypoint <code>pos[i_start]</code> . <code>step_dtheta</code> defines the actor's action as $\theta_{\text{now}} + \delta\theta$ , so referencing the nominal identically makes the two controllers' actions the same physical quantity — a prerequisite for one $Q$ scoring both. <b>Under vanilla this is not cosmetic:</b> an entire Action runs open-loop, so plan-versus-actual drift accumulates monotonically and <code>pos[i_end] - pos[i_start]</code> would bake that growing error into every stored action. For the indices, <code>result["position"]</code> is <code>[T, n_joints]</code> and the executor consumes exactly one row per <code>scene.step()</code> , so row index and physics step are 1:1; <b>at 25 Hz a tick is always 10 rows</b> , so for tick $m$ within an Action, $i_{\text{start}} = 10 * m$ and $i_{\text{end}} = i_{\text{start}} + 9$ , with no accumulator involved. $T$ covers one <code>play_once</code> Action, not the episode, so under vanilla $i$ runs $0 \dots T-1$ across the whole Action and does reach the thousands. Two guards: <b>clamp <math>i_{\text{end}}</math> to <math>T-1</math></b> , since the executor's $i < \_n\_wp(...)$ check means it holds the last target once the plan runs out; and <b>compute per arm</b> , since <code>left_arm</code> and <code>right_arm</code> are independent plan dicts with different $T$ . |
| Action vector layout: padding and prismatic columns | <code>MAX_ACTION = 18</code> . Width is $2 \times \text{spec.n\_joints}$ , so 16 for the three 6-DOF embodiments and 18 for franka-panda, right-padded with zeros, built by differencing already-padded <code>qpos_raw</code> and writing only at <code>joint_index</code> columns. Prismatic gripper columns stay exactly zero: <code>joint_index</code> is -1 for the gripper token so the actor never writes them, and <code>step_dtheta</code> ignores them since grippers hold their current opening. The HJ actor therefore never opens or closes the gripper; grasp timing stays with the task script.                                                                                                             | Same layout, but $a_{\text{nom}}$ must respect both properties. Because the padding applies to the concatenated two-arm vector, the right arm begins at column 8 for 6-DOF embodiments and column 9 for franka — <b>there is no fixed per-arm stride</b> , so assemble using the embodiment's own <code>spec.n_joints</code> and then pad. A hardcoded stride would silently write the right arm into the wrong columns for three of the four embodiments. The prismatic columns must also stay zero: the critic forms each link's Cartesian effect as <code>Jlin @ dtheta</code> summed over all columns and the prismatic Jacobian columns are non-zero, so a stray value there would make the critic attribute real motion to a component the environment will not execute. The planner's <code>position</code> array is arm-only, since grippers go through a separate <code>set_gripper</code> stream, so this is safe by default — the risk is only if $a_{\text{nom}}$ is built by differencing <code>qpos_raw</code> , which does include gripper channels.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Episode scheduling                                  | Three independent <code>rng.choice</code> calls in <code>sample_scene</code> for task, embodiment and obstacle; IID sampling over 25 episodes leaves the 20 (task, embodiment) combinations badly unbalanced. Every episode was <code>on_path</code> with no filter, so neither 80/20 split existed. A spawn failure returned ( <code>None</code> , <code>None</code> , <code>None</code> , <code>arm</code> ) and the episode simply ran with <code>env.obstacle = None</code> .                                                                                                                                                                                                                                         | A fixed cyclic schedule over the shuffled $5 \times 4$ product, so task and embodiment counts are exactly equal; the obstacle axis disappears since there is now one model. Two further draws are made independently per episode — obstacle placement (80% <code>on_path</code> / 20% <code>off_path</code> ) and behavior policy (80% with switching / 20% nominal only) — giving four episode types. Keeping them independent avoids confounding the filter engaging with the obstacle being in the way. Episodes skipped by the tier-3 spawn failure in section 1 must be re-drawn into the same schedule slot or the balance quietly breaks, and skips should be counted per round split by mode, since <code>off_path</code> is far more prone to failure than <code>on_path</code> given the 0.22 m lateral offset against <code>TABLE_XYLIM</code> bounds of $[-0.48, 0.48] \times [-0.32, 0.28]$ .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |



| Item                  | Previously                                                                                                                                                                                                                                                                                                                                          | To be done |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Planner's world model | make_training_env spawns the obstacle physically, then calls update_curobo_world(env.robot) with no obstacle argument, so cuRobo's world contains only the table. The expert plans a collision-free path as if the obstacle were absent and drives into it. There is no termination on violation, so the rollout keeps recording after penetration. | Unchanged. |

### 6. Buffer — main/train/buffer.py

| Item                                  | Previously                                                                                                                                                                                                                                                             | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| New fields: task_id and action_source | FIELDS stores embodiment_id as an int8 but nothing identifying the task, so per-task metrics cannot be computed offline at all. Action origin is likewise unrecorded, and since every action was an achieved displacement they are indistinguishable by source anyway. | Two int8 fields. task_id indexes the 5-task list, enabling free per-task splits of h, Q and the h - Q gap at every gradient step since it rides along in the sampled batch. action_source marks planner / actor / perturbation, letting us audit how much of the buffer sits in the actor's action box versus planner scale, and diagnose distribution shift as the DAgger rounds progress.                                                                                                               |
| Capacity and schema                   | cfg.n_episodes * (cfg.max_steps_per_episode + 2), sized for one pass: 25 x 402 ≈ 10,050 steps ≈ 6.3 GB. The header records shapes and capacity, and append refuses a shape change after step 0.                                                                        | Same formula with the new tick budget, n_episodes * 452, but sized across all collection rounds up front since open_memmap allocates at fixed capacity and append asserts against it. At ~630 KB per step — dominated by the [1200, 256] fp16 scene tokens at 614 KB — 200 episodes is roughly 57 GB. Adding fields, changing h_scale, redefining h, changing the control rate and changing the action convention each invalidate every existing buffer, so this starts from fresh collection regardless. |

### 7. Logging — main/train/hfunc.py, eval\_utils.py, train.py

| Item                                         | Previously                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | To be done                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Video panel                                  | PanelVideoWriter renders a 320 x 480 panel beside the observer frame, 960 x 480 output, showing t, step index, h, V, h - V, control mode as NOMINAL / FILTER,  δθ  as a fraction of max, and a scrolling h/V mini-plot of the last ~100 samples, at video_fps = 10.0 frames per simulated second — described in a code comment as every 2nd tick, true only at 20 Hz.                                                                                                                                                                                                                 | Adds, in cm: dL, dR, dLh, dRh formatted exactly as record.py does (f"{val*100:.1f}cm", "inf" when non-finite); the true_argmin link name; per-arm holding state with provenance (L=bread(contact) R=none(arm_open)); the plan stage from task._cehj_skill; the HJ block counter as tick-in-block out of 3 plus cumulative seconds since engagement; and  a_nom  / dθ_max. The frame rate also needs fixing: at 25 Hz every 2nd tick is 12.5 fps, so the constant and the comment disagree — replace with an explicit integer stride, every 2 ticks for 12.5 fps or every 5 for 5 fps. Recommend every 2, since intervention blocks are only 3 ticks long and a coarser stride could skip an entire engagement. |
| Distance plumbing and panel self-consistency | compute_h already calls distance_info(env) once per control tick and receives all these fields, but its returned diag keeps only h, d_block, d_table, d_held, d_arm_arm, per_link, true_argmin, holding and contact — the per-arm and per-held-object terms are dropped on the floor. Separately, because h = min(d_block, d_table) while the panel showed only obstacle distances, h sat well below all four displayed numbers whenever the gripper descended toward the table and the panel appeared to contradict itself; a d_table readout was going to be added for that reason. | Forward d_left, d_right, d_left_held, d_right_held, holding_left, holding_right, and the hold_debug fields L_source / R_source / L_fail / R_fail. <b>Zero extra simulator cost</b> — the expensive part, contact queries and hold detection, is already paid every tick. No d_table readout is needed once the table term is gone: h = d_block = min(dL, dR, dLh, dRh) is exactly the minimum of the four displayed distances at every frame, so the panel becomes self-consistent by construction and doubles as a check that the plumbing is correct.                                                                                                                                                        |
| Sampling rate of h                           | In the sweeps distance_info ran on every one of the 250 Hz scene.step() calls, about 2720 per episode at 2.6 ms each, so d_min was a minimum over the full trajectory.                                                                                                                                                                                                                                                                                                                                                                                                                | Now once per 25 Hz control tick, about 272 per episode — a 10x reduction rather than the 12.5x it would have been at 20 Hz. At 0.3 m/s the arm covers 12 mm between samples instead of 15 mm, so the upward bias in recorded minimum h shrinks slightly but does not go away. If it matters, keep a min-tracker on the 250 Hz hook and have the tick read the running minimum instead of an instantaneous sample.                                                                                                                                                                                                                                                                                              |
| Plan stage                                   | Already live and unused. ResidualController.attach() calls _install_skill_tracker(self.env.task), which stamps every play_once Action and keeps task._cehj_skill in sync, producing strings like "L pre_grasp bread   R lift dz=0.1".                                                                                                                                                                                                                                                                                                                                                 | Read it in _capture_tick and display it. No new machinery.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Camera streams                               | Observer camera only, via cameras.observer_camera.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Unchanged.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Training-loop metrics                        | grad_step returns global mean_q, mean_h, mean_gap_h_q, plus losses, grad norms, alpha, gamma and buffer size, with no per-embodiment or per-task breakdown.                                                                                                                                                                                                                                                                                                                                                                                                                           | The same three quantities additionally split by embodiment_id and the new task_id. Free, since both fields arrive in the sampled batch — a masked mean over existing tensors, no extra forward passes. <b>What cannot go here:</b> per-episode intervention, success and collision rates. Training is fully offline — grad_step samples a static memmap and never touches the simulator — so there are no episodes in the gradient loop to measure, and an offline "collision rate" would just be the fraction of dataset transitions with h < 0: a fixed property of the buffer that never improves, and actively misleading as a progress signal. Those ride the rollout-sweep cadence instead.              |
| Rollout sweeps and eval videos               | No debug rollouts during training. The only simulator contact is evaluate(), which runs one filtered episode on a random scene every eval_every = 1024 grad steps, writing a single video to runs/videos/eval_{mode}_{emb}_{task}_s{seed}_step{N}.mp4.                                                                                                                                                                                                                                                                                                                                | 5 rollouts per sweep, one per task, with the embodiment rotating across sweeps so all 20 combinations are covered every 4 sweeps. Single camera. Triggered at training start and every 10 epochs. Budget roughly 10-15 minutes per sweep — the earlier 8-12 minute estimate scaled up for 25% more ticks per episode. Eval matches: one video per task, same panel, same per-(task, embodiment) metrics.                                                                                                                                                                                                                                                                                                       |
| Success / collision / intervention           | Eval only, one random (task, embodiment, obstacle) triple per eval, reported as flat aggregates in trace["metrics"].                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Computed per (task, embodiment) from the rollout sweeps and held as an EMA, so the wandb panels stay populated between sweeps instead of going sparse. Calibration note: the obstacle-blind expert baseline is 17/64 success and 53/64 contact, so success rate is a noisy, low-baseline signal needing many episodes before it means anything.                                                                                                                                                                                                                                                                                                                                                                |

### 8. Diagnostics to add while we are in there

**Realized versus commanded displacement, per tick, as a ratio against dθ\_max.** We are now deliberately excluding tracking error from the stored action, so we should measure how large it is rather than assume it is small. This also quantifies the one inconsistency that survives the change: the two controllers have different intra-tick profiles for the same net command — cuRobo ramps through 10 intermediate targets while step\_dtheta sets a single target and holds it — so identical δθ values produce slightly different realized motion depending on who issued them. If that turns out to be large, Q(s, a) is mildly controller-dependent and we would want to build a\_nom from a one-tick forward simulation of the drives rather than the raw spline.

**Mode-switch count per episode.** Hysteresis is gone and the commitment block is only 3 ticks, so the filter can legally flip every 0.12 s. This is the number that tells us whether that matters in practice. If it is high, lengthen hj\_hold\_ticks rather than reintroducing a second margin — the block is doing the same job with one fewer parameter.

**Action-magnitude histogram by action\_source.** The actor is bounded to κ = 0.25 of each joint's velocity limit per tick, while cuRobo plans against the full limit, so a\_nom can exceed dθ\_max several times over by construction. Tracking the distribution per source shows how much of the critic's evaluation region is covered by data versus extrapolated, and whether the DAgger rounds are filling in the actor's small-action region.

**Fraction of ticks with an arm link below the table plane.** Cheap to compute and no longer part of h, but it is the direct measurement of the risk accepted by removing the table term. If the actor starts driving links through the tabletop this is the number that will show it first, well before it shows up as a strange-looking video.

PART II — Frozen-encoder parity with HoloBrain (found in the RoboOrchardLab scan, after Part I was written)

Part I assumes the frozen HoloBrain-0 encoder is being driven correctly. This part checks that assumption and finds it is not. We reuse HoloBrain-0 as a frozen encoder, so our observation dict must be constructed the way their inference pipeline constructs it; every mismatch below is silent — no exception, just degraded scene tokens feeding the trunk, which the geometric trunk then treats as metrically true. Nothing in Part I is worth running until these are settled. All paths are relative to `/storage1/fs1/sibai/Active/yuxuan/cross_embodiment/`, with **OURS** = **CEHJ**/ and **HB** = **RoboOrchardLab**/. Each row ends with whether I verified it directly in the source or am relaying it from the automated scan.

9. Observation and encoder-input mismatches

| Item                                              | HoloBrain reference (what it expects)                                                                                                                                                                                                                                                                                                                                                                                                                                               | CEHJ today (what we send)                                                                                                                                                                                                                    | Why it matters / what to do                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Image resolution                                  | HB/projects/holobrain/configs/config_holobrain_gd_common.py:43 sets dst_wh=(320, 256). The deploy transform Resize (HB/projects/holobrain/configs/config_robotwin_dataset.py:171-174) resizes every camera to it and <b>rescales the intrinsics</b> by the same factors (HB/robo_orchard_lab/dataset/robotwin/transforms.py:493-500).                                                                                                                                               | OURS/main/envs/env.py get_encoder_obs() uses the D435 native 320x240 (OURS/RoboTwin/env_cfg/task_config/_camera_config.yml:11-14) with <b>no resize</b> and no intrinsic rescale. image_wh is taken from the raw frame at env.py:470.        | Height is 240 where the model expects 256, so fy/cy are off by 256/240 and the feature grid has the wrong number of rows. <b>Add the resize + intrinsic rescale to get_encoder_obs.</b> Verified directly.                                                                                                                                                                                                                                                                                                                                                                                                          |
| Scene-token count (proof of the above)            | The four pyramid levels are strides ( <b>8, 16, 32, 64</b> ), stated outright as depth_gt_stride under multi_task=True (HB/.../config_holobrain_gd_common.py:131-136) and consistent with Swin out_indices=(1,2,3) plus ChannelMapper(num_outs=4) (:171, :183). Both sides select feature_level = [1, 2] (HB/.../config_holobrain_gd_common.py:372), i.e. <b>strides 16 and 32</b> . At 320x256 that is 20x16 + 10x8 = 400 tokens per camera; with 3 cameras, exactly <b>1200</b> . | OURS/main/train/buffer.py:37-38 hardcodes scene_tokens (1200, 256) and scene_pos (1200, 3) — no padding constant, unlike MAX_STATE_TOKENS / MAX_ACTION. OURS/main/network/encoder.py:387 uses the same feature_level=(1, 2).                 | 1200 is only reachable at 320x <b>256</b> with 3 cameras. At our 320x240 the count is 20x15 + 10x8 = 380 per cam = <b>1140</b> , and append would assert. Collection has never run (CKPT_DIR unreachable), so this has stayed latent. <b>This constant is the proof that the resize is missing.</b> Verified directly.                                                                                                                                                                                                                                                                                              |
| projection_mat coordinate frame                   | GetProjectionMat(target_coordinate="ego") (HB/.../config_robotwin_dataset.py:178) builds P = K @ T_world2cam @ T_base2world @ inv(T_base2ego) (HB/robo_orchard_lab/dataset/robotwin/transforms.py:847-871). MoveEgoToCam sets T_base2ego from the <b>last</b> camera in cam_names (transforms.py:63-65), i.e. the head. So 3D lifting happens in <b>head-camera (ego) frame</b> .                                                                                                   | OURS/main/envs/env.py:453-457 builds P = K @ T_world2cam and stops, so lifting is in <b>world frame</b> . The get_encoder_obs docstring states this is deliberate ("the encoders and body features stay in the world frame").                | <b>Highest-risk item.</b> The spatial enhancer back-projects depth with P and then embeds the resulting 3D coordinates with weights trained on ego-frame values; world coordinates are offset by the base pose, so the learned position embedding runs far out of distribution — silently. <b>Fix that keeps both properties:</b> feed the encoder ego-frame P (matching training), then transform the returned positions to world frame ourselves for the trunk. Verified directly.                                                                                                                                |
| Extrinsic direction and naming                    | Field is t_world2cam (HB/robo_orchard_lab/models/holobrain/processor.py:58-60). The raw RoboTwin path uses extrinsic_cv <b>directly</b> , no inverse (HB/projects/holobrain/policy/robotwin_policy.py:111-115); the formatted path inverts a camera-in-world pose (robotwin_policy.py:159-190).                                                                                                                                                                                     | OURS/main/envs/env.py:391-402 camera_extrinsic() returns cam.get_extrinsic_matrix(), which is also world→camera. The obs key is named T_cam_world (env.py:472).                                                                              | <b>Direction MATCHES</b> — this is the one geometry convention we got right. Only the *name* is inverted relative to its semantics; rename to T_world2cam to stop it misleading the next reader. Verified directly.                                                                                                                                                                                                                                                                                                                                                                                                 |
| Camera set and ordering                           | Two processors exist: a 4-camera list [front, left, right, head] (HB/.../config_robotwin_dataset.py:55-60) and a 3-camera robotwin2_0_ur5_wsg list [left_camera, right_camera, head_camera] (:112). The processor stacks strictly in that order (HB/robo_orchard_lab/models/holobrain/processor.py:118-143).                                                                                                                                                                        | OURS/main/envs/env.py:441-445 uses [head, left_wrist, right_wrist], labelled at env.py:477. ARX-X5 / piper configs have front_camera commented out (OURS/RoboTwin/assets/embodiments/ARX-X5/config.yml:27-40), so only 3 exist.              | The 1200-token arithmetic implies <b>3 cameras</b> , so the ur5_wsg processor is our reference — but its order puts head <b>last</b> and ours puts it <b>first</b> . Camera index is preserved through backbone and spatial enhancer with no name-based reordering, and the ego anchor is the last camera. <b>Reorder to [left, right, head]</b> . Arithmetic verified; which processor the v0.0_GD checkpoint exports is undetermined without the checkpoint on disk.                                                                                                                                              |
| feature_enhancer / text branch                    | GD config sets multi_task=True (:45), which builds a TextImageDeformable2DEnhancer (:202) and a text encoder. BIP3D._forward runs text extraction → <b>feature_enhancer</b> → spatial_enhancer → decoder (HB/robo_orchard_lab/models/bip3d/structure.py:131-166). The processor maps a missing instruction to "", which is still tokenized (processor.py:145-147).                                                                                                                  | OURS/main/network/encoder.py:246-260 loads only backbone, neck, backbone_3d, neck_3d, spatial_enhancer and decoder.robot_encoder. There is <b>no</b> feature_enhancer, no text_encoder, and no instruction anywhere in get_encoder_obs.      | We run correctly-loaded spatial-enhancer weights on inputs that never passed the fusion stage they were trained after. Dropping language from a safety filter is defensible; feeding a frozen module out-of-distribution inputs is a separate question. <b>Decide empirically:</b> either load and run feature_enhancer with a fixed instruction, or measure the token-distribution shift and accept it. Verified directly.                                                                                                                                                                                         |
| Depth acquisition and masking                     | HB/projects/holobrain/policy/robotwin_policy.py:109 takes camera_data["depth"] / 1000. That observation is built by OURS/RoboTwin/envs/camera/camera.py:412-418, depth_image = -position[...,:2] * 1000.0, and every consumer line then applies res[cam]["depth"] *= rgba[cam]["rgba"][[:,:3] / 255 (:433-434, :441, :445). Net input to the model is therefore <b>-pos[...,:2] x (alpha/255)</b> in metres.                                                                        | OURS/main/envs/env.py:451-452 reads cam.get_picture("Position") directly and takes -pos[:, : , 2] in metres. No /1000 needed, but also <b>no alpha mask</b> .                                                                                | Units, sign and shape agree — the round trip x1000 then /1000 cancels, so no scale bug and no uint16 quantization (they cast to float64). The single difference is the <b>alpha factor we omit</b> : background pixels are exactly 0 for them and whatever the position buffer holds for us, and partially transparent materials get their depth scaled down. Depth feeds backbone_3d and hence feature_3d, so this shifts the enhancer's inputs rather than being cosmetic. <b>Multiply by alpha/255 in</b> get_encoder_obs, taking alpha from the same Color picture as the RGB. Verified directly on both sides. |
| Normalization and channel order                   | ImageChannelFlip([2,1,0]) in deploy (config_robotwin_dataset.py:175) then BaseDataPreprocessor(channel_flip=True, mean=[123.675,116.28,103.53], std=[58.395,57.12,57.375]) (config_holobrain_gd_common.py:140-157), applied on float32 <b>0-255</b> with HWC→CHW (HB/robo_orchard_lab/models/layers/data_preprocessors.py:116-157).                                                                                                                                                 | OURS/main/envs/env.py:386-388 defines the same mean/std and applies them at env.py:463-464 on float32 0-255, permuting to [B, n_cam, 3, H, W] at :468. No channel flips.                                                                     | <b>Effectively MATCHES</b> for live SAPIEN RGB: their two flips cancel, so both end up RGB with identical mean/std and layout. No change needed — but do not feed BGR anywhere, since we have no flip to compensate. Verified directly.                                                                                                                                                                                                                                                                                                                                                                             |
| joint_state source                                | robotwin_policy.py:123-128 passes joint_action["vector"], which RoboTwin builds from get_left_arm_jointState() / get_right_arm_jointState() — these read <b>drive targets</b> (OURS/RoboTwin/envs/robot/robot.py:510-514).                                                                                                                                                                                                                                                          | OURS/main/envs/env.py:465-475 uses get_left_arm_real_jointState() / get_right_arm_real_jointState(), which read <b>measured qpos</b> (RoboTwin/envs/robot/robot.py:524-531).                                                                 | The robot-state encoder was trained on commanded values; we feed achieved ones. Same commanded-vs-achieved theme as the action side, and largest exactly during contact. Layout, radians and the trailing normalized gripper scalar all match. <b>Switch to the drive-target getters.</b> Verified directly.                                                                                                                                                                                                                                                                                                        |
| joint_scale_shift / joint_mask / embodiedment_mat | The decoder applies apply_scale_shift(hist_robot_state, joint_scale_shift) (HB/robo_orchard_lab/models/holobrain/action_decoder.py:445-447); deploy sets joint_mask = ([True]*6 + [False])*2 so gripper joints are masked in the robot encoder; FK uses embodiedment_mat = T_base2ego from GetProjectionMat.                                                                                                                                                                        | OURS/main/network/encoder.py:308-311 calls encode_robot_state(robot_state, kinematics.joint_relative_pos, joint_mask) with raw angles, <b>no</b> scale/shift, and joint_mask is never populated by the rollout. No embodiedment_mat is used. | Three separate omissions on the same path, all silent. Couples to the ego-frame item above. <b>Verify against the checkpoint's exported processor JSON before changing anything</b> — reported by scan, not re-verified by me.                                                                                                                                                                                                                                                                                                                                                                                      |

| Item                         | HoloBrain reference (what it expects)                                                                                                                                                                                                                   | CEHJ today (what we send)                                                                                                                                                                                  | Why it matters / what to do                                                                                                                                                                                                                    |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model loading and invocation | HoloBrainInferencePipeline.load_pipeline(..., load_impl="native", model_prefix="model") (HB/projects/holobrain/scripts/robotwin_eval.py:163-170) builds the whole graph from model.config.json, then pipeline.model.eval() and @torch.inference_mode(). | OURS/main/network/encoder.py:246-260 calls safetensors.torch.load_file and strips prefixes into a partial wrapper, then .to(device, dtype).eval(); every encode_* is @torch.no_grad(). Default dtype fp32. | eval mode, no-grad and fp32 all <b>MATCH</b> (their GD path has no autocast; drop_rate = 0 so no stochastic layers). The only real difference is the subset of modules loaded, already covered by the feature_enhancer row. Verified directly. |

### 10. How to settle all of Part II at once

**Write one parity test.** Build a single observation two ways from the same simulator state — once through HoloBrain's own processor (HB/robo\_orchard\_lab/models/holobrain/processor.py plus the deploy transform chain in HB/projects/holobrain/configs/config\_robotwin\_dataset.py) and once through our OURS/main/envs/env.py::get\_encoder\_obs — then assert tensor-by-tensor agreement on imgs, depths, image\_wh, projection\_mat and joint\_state. That converts every row above from an argument into a pass/fail, including the three I could not verify statically, and it will catch whichever processor variant the v0.0\_GD checkpoint actually exports. It needs a reachable CKPT\_DIR, which is the blocker already listed at the top of Part I.

**Priority order.** projection\_mat frame first (it corrupts every token position, and the trunk consumes those as metric truth); then image resolution together with the 1200-token constant, since they are the same bug and one of them will hard-fail on the first append; then the missing feature\_enhancer; then camera ordering; then the joint-state source. The zero-velocity actor bug in Part I section 3 is real but sits below all of these — it corrupts the action, whereas these corrupt the state.